CS353: Computer Networks — Lab 12 / Lab 11

Quality of Service at a Router: Written Report & Answers

Course: CS353 · Network Layer / Congestion and QoS

Assignment: Quality of Service at a Router

1. Executive Summary of Implementation

This submission provides the implementation and complete verification of the four core Quality-of-Service (QoS) mechanisms applied by a router to traffic flows sharing a single bottleneck output link of capacity *C = 10 Mbps*:

1. Admission Control: Regulates which flows enter the network using the Parekh-Gallagher test.

2. Traffic Policing: Enforces a traffic contract using a Token Bucket, dropping non-conforming packets.

3. Traffic Shaping: Smooths traffic bursts using the same Token Bucket, buffering excess packets to eliminate loss.

4. Packet Scheduling: Arbitrates link bandwidth across admitted flows using Weighted Fair Queuing (WFQ), contrasted with First-In First-Out (FIFO).

2. Part A: Admission Control (Written Answers)

Experimental Setup:

Shared Link Capacity: *C = 10 Mbps*

Admission Criterion (Parekh-Gallagher): A candidate flow *i* is admissible iff:

$$(W_j / \Sigma W) \cdot C \geq R_j$$

holds for **every** admitted flow *j* after adding *i*, where *ΣW* is the total weight of all admitted flows including *i*.

Sequential Processing Table:

Flow Request	Weight (*W*)	Requested Rate (*R*)	Candidate *ΣW*	Guaranteed Rate	Decision
f1	1	2.0 Mbps	1	*(1/1) × 10 = 10.0 Mbps*	**ADMITTED**
f2	1	2.0 Mbps	2	*(1/2) × 10 = 5.0 Mbps*	**ADMITTED**
f3	2	4.0 Mbps	4	*(2/4) × 10 = 5.0 Mbps*	**ADMITTED**
f4	1	3.0 Mbps	5	*(1/5) × 10 =* ***2.0 Mbps***	**REJECTED**

Confirmation: Flows f_1, f_2, and f_3 are admitted (*ΣW = 4*). Flow f_4 requests *3.0 Mbps* but would only receive a guaranteed rate of *2.0 Mbps*, and is thus rejected.

Written Answer A1: Which flow's guarantee does the rejection protect?

The rejection of flow f_4 protects **two distinct guarantees**:

1. Flow f_4's Own Contractual Guarantee: Under QoS admission control, the network must never accept a flow whose service-level agreement (SLA) cannot be honored. Because f_4 requests *3.0 Mbps* but its weighted share guarantees only *2.0 Mbps*, admitting f_4 would immediately breach the contract promised to f_4.

2. The Contractual Guarantees of Previously Admitted Flows (f_1, f_2, f_3):

The total rate already committed to active flows is:

$$R_1 + R_2 + R_3 = 2.0 + 2.0 + 4.0 = 8.0 \text{ Mbps}$$

On a link of capacity $C = 10.0$ Mbps, only $10.0 - 8.0 = 2.0$ Mbps of spare capacity remains. If the router attempted to force f_4 to receive its requested 3.0 Mbps by inflating its weight (e.g. setting $W_4 = 2$, yielding $\Sigma W = 6$), the bandwidth allocated to existing flows would drop:

- f_1 guaranteed rate would fall to $(1/6) \times 10 = 1.67$ Mbps < 2.0 Mbps (Violation)
- f_2 guaranteed rate would fall to $(1/6) \times 10 = 1.67$ Mbps < 2.0 Mbps (Violation)
- f_3 guaranteed rate would fall to $(2/6) \times 10 = 3.33$ Mbps < 4.0 Mbps (Violation)

Therefore, rejecting f_4 safeguards active flows f_1, f_2, and f_3 from bandwidth degradation and SLA violations.

Written Answer A2: What weight or rate would f_4 need to be admitted?

1. Rate Requirement:

With its assigned weight $W_4 = 1$, the candidate total weight is $\Sigma W = 4 + 1 = 5$. The guaranteed rate provided to f_4 is:

$$\text{Rate}_{f4} = (1 / 5) \times 10 \text{ Mbps} = 2.0 \text{ Mbps}$$

Therefore, flow f_4 would be admitted if its requested rate is reduced to:

$$R_4 \leq 2.0 \text{ Mbps}$$

2. Weight Requirement:

No weight assignment can admit f_4 at its requested rate of $R_4 = 3.0$ Mbps while preserving the guarantees of f_1, f_2, and f_3. This is because the sum of all four requested rates:

$$\Sigma R_i = 2.0 + 2.0 + 4.0 + 3.0 = 11.0 \text{ Mbps} > C = 10.0 \text{ Mbps}$$

exceeds the physical capacity of the link. Since the sum of guaranteed rates cannot exceed C, it is mathematically impossible to satisfy all four flows simultaneously.

---

3. Part B: Traffic Policing (Written Answers)

Experimental Setup:

Token fill rate: $r = 1.0$ token/time unit

Bucket capacity: $b = 4.0$ tokens (starts full)

Packet cost: $L = 1.0$ token per packet

Packet arrival trace: 6 packets at $t = 0$, 1 packet at $t = 5$, 1 packet at $t = 6$

Policing Execution Results:

Packet #	Arrival Time (t)	Tokens Before Arrival	Action	Tokens After Action
1	0.0	4.0	**PASS**	3.0
2	0.0	3.0	**PASS**	2.0
3	0.0	2.0	**PASS**	1.0
4	0.0	1.0	**PASS**	0.0
5	0.0	0.0	**DROP**	0.0
6	0.0	0.0	**DROP**	0.0
7	5.0	4.0	**PASS**	3.0
8	6.0	4.0	**PASS**	3.0

Result: 6 packets pass, 2 packets are dropped.

Written Answer B: Explain why exactly two packets are dropped, in terms of bucket capacity.

The token bucket capacity is $b = 4$ tokens, and the bucket begins full.

At time $t = 0$, an instantaneous burst of 6 packets arrives simultaneously. Because the elapsed time between packets within the burst is zero ($\Delta t = 0$), no new tokens are generated during the burst.

Each packet requires $L = 1$ token. The first 4 packets consume all 4 tokens accumulated in the bucket ($4 - 4 = 0$). When the 5th and 6th packets arrive, the bucket has 0 tokens available.

Because traffic policing operates **without memory or buffering**, it cannot store non-conforming packets for later transmission. The router immediately discards any packet that arrives when tokens are insufficient. Therefore, the number of dropped packets equals the burst size minus the bucket capacity:

$$\text{Packets Dropped} = 6 - b = 6 - 4 = 2 \text{ packets}$$

---

4. Part C: Traffic Shaping (Written Answers)

Experimental Setup:

Same token bucket parameters ($r = 1.0$, $b = 4.0$, starts full) and identical arrival trace.

Non-conforming packets are queued in a FIFO buffer and released as soon as sufficient tokens accrue.

Shaping Execution Results:

Packet #	Arrival Time (t_{arr})	Release Time (t_{rel})	Delay ($t_{rel} - t_{arr}$)	Status
1	0.0	0.0	0.0	Delivered
2	0.0	0.0	0.0	Delivered
3	0.0	0.0	0.0	Delivered
4	0.0	0.0	0.0	Delivered
5	0.0	1.0	1.0	Delivered (Buffered)
6	0.0	2.0	2.0	Delivered (Buffered)
7	5.0	5.0	0.0	Delivered
8	6.0	6.0	0.0	Delivered

Result: Zero packets dropped (*100\%* delivery rate).

Written Answer C1: Report packet delays and compare with Part B.

1. Delays in Shaping:

- Packets 1 to 4: Delay = **0.0** (released immediately at $t = 0$)
- Packet 5: Delay = **1.0** time unit (released at $t = 1.0$)
- Packet 6: Delay = **2.0** time units (released at $t = 2.0$)
- Packets 7 and 8: Delay = **0.0** (released immediately at $t = 5.0$ and $t = 6.0$)

2. Comparison with Part B (Policing):

- In Part B (Policing), packets 5 and 6 experienced **complete packet loss** (dropped permanently; infinite delay).
- In Part C (Shaping), packet loss was **zero**. Packets 5 and 6 experienced minor, bounded queuing delays (*1.0* and *2.0* time units) while awaiting token regeneration, resulting in a smoothed, conforming egress stream.

Written Answer C2: State the single difference between shaping and policing that produces drops in one and delays in the other.

The single fundamental architectural difference is the **presence of a Packet Buffer (Queue)**:

Traffic Policing has NO buffer: Packets arriving when the token bucket has fewer than *L* tokens are immediately discarded (or marked).

Traffic Shaping HAS a buffer: Packets arriving when tokens are insufficient are placed into a queue and held until enough tokens accrue at rate *r*. Shaping trades buffer delay for zero packet loss.

---

5. Part D: Packet Scheduling (Written Answers)

Admitted Flows (Carried from Part A):

Flow 1: weight $w_1 = 1$

Flow 2: weight $w_2 = 1$

Flow 3: weight $w_3 = 2$

Finish Number Formulation:

For packet k of flow i with arrival A_k, length L_k, and weight w_i:

$$F_k = \max(A_k, F_{k-1}) + (L_k / w_i) \quad (\text{with } F_0 = 0)$$

Finish Number Computations:

Packet	Flow	Weight (*w*)	Arrival (*A*)	Length (*L*)	Start Virtual	Finish Number (*F*)
A	Flow 1	1	0	8	0	*max(0, 0) + 8/1 =* ***8.0***
C	Flow 3	2	5	10	5	*max(5, 0) + 10/2 =* ***10.0***
B	Flow 2	1	5	6	5	*max(5, 0) + 6/1 =* ***11.0***
E	Flow 3	2	8	8	10	*max(8, 10) + 8/2 =* ***14.0***
F	Flow 1	1	10	6	10	*max(10, 8) + 6/1 =* ***16.0***
G	Flow 3	2	11	10	14	*max(11, 14) + 10/2 =* ***19.0***
D	Flow 2	1	8	9	11	*max(8, 11) + 9/1 =* ***20.0***
H	Flow 2	1	20	8	20	*max(20, 20) + 8/1 =* ***28.0***

Output Orders:

Weighted Fair Queuing (WFQ) Order:

$$A \longrightarrow C \longrightarrow B \longrightarrow E \longrightarrow F \longrightarrow G \longrightarrow D \longrightarrow H$$

First-In First-Out (FIFO) Order:

$$A \longrightarrow B \longrightarrow C \longrightarrow D \longrightarrow E \longrightarrow F \longrightarrow G \longrightarrow H$$

Written Answer D1: Compare the two orders and explain how the weight-2 flow is served.

1. Order Comparison:

- Under FIFO, packets are served purely in order of arrival timestamps. Packet B and C both arrive at *t=5*, and D and E both arrive at *t=8*. FIFO transmits Flow 2's packets B and D ahead of or alongside Flow 3's packets.

- Under WFQ, transmission order is governed by the virtual finish number *F*, which scales inversely with weight: *(L / w)*.

2. How Flow 3 (Weight 2) is Served:

- Because Flow 3 has weight $w_3 = 2$, its transmission duration in virtual time is halved compared to Flows 1 and 2 ($w = 1$).

- When Packet C ($L=10$) and Packet B ($L=6$) arrive together at $t=5$, Packet C's finish number is $5 + 10/2 = 10.0$, while Packet B's finish number is $5 + 6/1 = 11.0$. Therefore, **Packet C is transmitted before Packet B**, despite being larger!

- Packet E arrives at $t=8$ with length 8, finishing at $10 + 8/2 = 14.0$, allowing it to jump ahead of Flow 1's Packet F ($F=16.0$) and Flow 2's Packet D ($F=20.0$).

- Packet G arrives at $t=11$ and finishes at $14 + 10/2 = 19.0$, jumping ahead of Flow 2's Packet D ($F=20.0$, arrived earlier at $t=8$).

- Overall, Flow 3 receives **50% of the link capacity**, draining twice as fast as Flows 1 and 2 (each receiving 25%).

Written Answer D2: How does Weighted Fair Queuing protect a flow from being starved by another?

1. Vulnerability in FIFO: In a FIFO scheduler, an aggressive, non-conforming flow that floods the queue with packets will monopolize the queue and output link. Any delay-sensitive or well-behaved flow arriving behind this burst is starved of bandwidth.

2. Protection Mechanism in WFQ:

WFQ isolates flows by maintaining per-flow virtual finish tracking:

$$F_k = \max(A_k, F_{k-1}) + (L_k / w_i)$$

- If an aggressive flow pumps back-to-back packets into the queue, its F_{k-1} accumulates continuously into the virtual future ($F = 50, 80, 120\dots$).

- When a well-behaved or previously idle flow transmits a packet at time *A*, its finish number starts fresh from its arrival time:

$$F_{idle} = A + (L / w)$$

- Because its finish number is much smaller than the inflated finish numbers of the flooded flow, the scheduler immediately serves the conforming flow's packet ahead of the backlogged packets of the aggressive flow.

3. Guaranteed Minimum Share: WFQ mathematically guarantees that every flow *i* receives at least its weighted link share $(w_i / \Sigma w) \cdot C$ over any active window, ensuring **starvation is impossible**.

---

6. Program Verification Output Reference

The Python implementation [`qos_router.py`](qos_router.py) executes all four mechanisms, matches all PDF-specified test results, and prints the exact verification output stored in [`program_output.txt`](program_output.txt).